HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
VIỆN KHOA HỌC KỸ THUẬT BƯU ĐIỆN

BÁO CÁO NỘI DUNG

Nội dung 3: Xây dựng module lõi xử lý âm thanh và nhận dạng tiếng nói

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN NĂM 2026

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG MEET-ASSISTANT: TỰ ĐỘNG HÓA TAKE-NOTE VÀ ĐỒNG BỘ HÓA QUẢN LÝ CÔNG VIỆC

18-SV-2026-RIPT

Thuộc nhóm ngành khoa học: Công nghệ thông tin

Chủ trì đề tài: Trần Đức Định (B23DCCC038)
Thành viên: Hoàng Nguyễn Trà My (B23DCCC116)
Giảng viên hướng dẫn: TS. Lê Văn Hoàng

Hà Nội – 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU NỘI DUNG.....1
1.1. Nhiệm vụ theo đề cương.....1
1.2. Phương pháp thực hiện.....1
CHƯƠNG 2. XÂY DỰNG HỆ THỐNG.....2
2.1. Môi trường và công cụ phát triển.....2
2.2. Xây dựng dịch vụ lõi core-service.....3
2.2.1. Tổ chức mã nguồn và phân quyền.....3
2.2.2. Outbox, publisher và chống xử lý lặp.....4
2.2.3. Điều phối luồng xử lý cuộc họp.....6
2.3. Ba đường thu nhận nội dung cuộc họp.....6
2.3.1. Tải tệp ghi âm lên MinIO.....7
2.3.2. Bot tham gia và ghi âm Google Meet.....7
2.3.3. Tiện ích mở rộng đọc phụ đề.....9
2.4. Mô-đun nhận dạng tiếng nói.....10
2.5. Mô-đun tách người nói.....13
2.6. Triển khai và vận hành.....14
CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ.....16
3.1. Môi trường và bộ dữ liệu thực nghiệm.....16
3.1.1. Môi trường.....16
3.1.2. Kịch bản cuộc họp có đáp án.....16
3.1.3. Dựng âm thanh bằng giọng tổng hợp.....17
3.2. Độ đo đánh giá.....17
3.3. Kết quả nhận dạng tiếng nói.....18
3.4. Kết quả tách người nói và phát hiện nói chồng.....20
3.5. Thời gian xử lý, chi phí và hiệu quả tiết kiệm thời gian.....21
3.5.1. Thời gian xử lý.....21
3.5.2. Chi phí.....22
3.5.3. Ước tính hiệu quả tiết kiệm thời gian.....22
CHƯƠNG 4. KẾT QUẢ NỘI DUNG VÀ ĐÁNH GIÁ.....24
4.1. Đối chiếu với kết quả cần đạt.....24
4.2. Tồn tại và hướng hoàn thiện.....24
TÀI LIỆU THAM KHẢO.....26

DANH MỤC BẢNG BIỂU

Bảng 2.1. Ngôn ngữ, framework và thư viện chính dùng để xây dựng hệ thống.....2
Bảng 3.1. Môi trường thực nghiệm.....16
Bảng 3.2. Các điều kiện âm thanh thử nghiệm.....17
Bảng 3.3. Kết quả nhận dạng tiếng nói theo điều kiện âm thanh.....18
Bảng 3.4. Kết quả tách người nói (Whisper small).....20
Bảng 3.5. Thời gian xử lý bản ghi 151 giây ở điều kiện sạch.....22
Bảng 3.6. Ước tính thời gian công sức con người cho một cuộc họp 60 phút.....22
Bảng 4.1. Đối chiếu kết quả Nội dung 3 với đề cương.....24

DANH MỤC HÌNH VẼ

Hình 2.1. Phân quyền theo vai trò kết hợp phạm vi tài nguyên.....4
Hình 2.2. Sơ đồ tuần tự sự kiện giữa core-service và ai-worker.....5
Hình 2.3. Luồng xử lý từ âm thanh đến công việc.....6
Hình 2.4. Ba đường thu nhận nội dung cuộc họp hội tụ tại transcript_versions.....7
Hình 2.5. Trạng thái một phiên bot và các lý do kết thúc.....8
Hình 2.6. Kiến trúc mô hình Whisper với chuỗi token đặc biệt điều khiển ngôn ngữ, nhiệm vụ và mốc thời gian.....11
Hình 2.7. Phiên âm hai lượt theo ngôn ngữ của từng đoạn.....12
Hình 2.8. Quy trình tách người nói bằng gom cụm vector đặc trưng giọng.....13
Hình 2.9. Triển khai hệ thống bằng Docker Compose: các container và volume.....14
Hình 3.1. Tỷ lệ lỗi từ (đã chuẩn hoá số) theo điều kiện âm thanh và cấu hình.....19
Hình 3.2. Thời gian xử lý từng bước cho bản ghi 151 giây (CPU, không GPU).....21

CHƯƠNG 1. GIỚI THIỆU NỘI DUNG

1.1. Nhiệm vụ theo đề cương

Nội dung 3 được thực hiện trong tháng 07–08/2026. Theo đề cương, nhiệm vụ của bước này là xây dựng module lõi xử lý âm thanh và nhận dạng tiếng nói: triển khai dịch vụ thu âm, xử lý nhiễu và chuyển đổi hội thoại thành văn bản. Kết quả cần đạt gồm API hoàn chỉnh cho xử lý âm thanh (Audio Processing) và module nhận dạng tiếng nói (STT) hoạt động ổn định với độ chính xác cao trên tiếng Việt.

Trong quá trình thực hiện, phạm vi của nội dung này được mở rộng theo kết quả tổng quan: ngoài phiên âm, hệ thống cần biết *ai nói câu nào* (tách người nói) để giao việc đúng người, và cần nhiều đường đưa âm thanh vào hệ thống (tải tệp, bot tham gia Google Meet, tiện ích đọc phụ đề). Phần nền của dịch vụ lõi (điều phối job, hàng đợi thông điệp, lưu trữ âm thanh) được xây dựng trước trong tháng 07–08/2026; các mô hình nhận dạng và tách người nói thật được gắn vào thay cho bộ giả lập trong giai đoạn hoàn thiện, khi giao diện chung đã ổn định.

1.2. Phương pháp thực hiện

Nhóm xây dựng theo nguyên tắc "thông luồng trước, thay mô hình sau": dựng toàn bộ đường đi của âm thanh với bộ phiên âm giả lập để kiểm chứng điều phối, sau đó thay bằng Whisper và mô hình giọng nói thật sau một giao diện chung, chọn bằng biến môi trường. Mỗi lựa chọn kỹ thuật (mô hình, tham số giải mã, ngưỡng gom cụm) được kiểm chứng bằng kiểm thử đơn vị và bằng thực nghiệm định lượng trên bộ dữ liệu dựng có đáp án, trình bày ở chương đánh giá của báo cáo này.

CHƯƠNG 2. XÂY DỰNG HỆ THỐNG

2.1. Môi trường và công cụ phát triển

Hệ thống gồm năm dịch vụ backend, một ứng dụng web và một tiện ích mở rộng trình duyệt. Dịch vụ lõi (core-service), cổng API (api-gateway) và dịch vụ tích hợp (integration-service) viết bằng TypeScript trên NestJS [11]; api-gateway hiện mới là khung dựng từ mẫu NestJS, còn integration-service đã có mô-đun đồng bộ công việc sang Trello và Telegram. Phần xử lý AI (ai-worker và ai-service, dùng chung mã nguồn) và bot ghi âm (bot-worker) viết bằng Python vì hệ sinh thái thư viện phù hợp. Bảng Bảng 2.1 liệt kê công cụ chính với phiên bản lấy từ `package.json`, `requirements.txt` và `Dockerfile`.

Bảng 2.1. *Ngôn ngữ, framework và thư viện chính dùng để xây dựng hệ thống*

Thành phần	Công nghệ và phiên bản	Vai trò
core-service	Node.js 22, TypeScript 5.8.3, NestJS 11.1.3	Khung ứng dụng, module, guard
core-service	Sequelize 6.37.7, sequelize-typescript 2.1.6	Truy cập PostgreSQL
core-service	@golevelup/nestjs-rabbitmq 6.0.1, minio 8.0.5	Sự kiện RabbitMQ; ghi object, ký URL
core-service	@nestjs/jwt 11.0.0, bcryptjs 3.0.2	Xác thực JWT, băm mật khẩu
integration-service	NestJS 11.1.3, @golevelup/nestjs-rabbitmq 6.0.1, Jest 30.0.0	Nhận sự kiện, gọi Trello REST API và Telegram Bot API, kiểm thử
ai-worker	Python 3.12.3, aio-pika 9.5.5, httpx 0.26.0	Nhận việc từ RabbitMQ, gọi API LLM
ai-worker	faster-whisper 1.2.1, PyAV 18.1.0	Nhận dạng tiếng nói, giải mã âm thanh
ai-worker	sherpa-onnx 2.0.1, NumPy 2.2.5	Vector đặc trưng giọng, gom cụm
ai-service	FastAPI 0.115.12, psycopg2 2.9.10, pypdf 6.21.4, python-docx 1.2.0	API hỏi đáp nội bộ, truy vấn pgvector, tách chữ tài liệu
bot-worker	Playwright 1.64.1 (Chromium), Xvfb, PulseAudio	Vào Google Meet trên màn hình và loa ảo
Extension	Chrome Manifest V3, node:test	Đọc phụ đề Meet, kiểm thử
Frontend	Next.js 16.3.8, React 19.2.8, Tailwind CSS 4.3, shadcn/Radix UI	Ứng dụng web
Hạ tầng	PostgreSQL 15 (pgvector 0.8.6), RabbitMQ 3.11, Redis 7, MinIO	Lưu trữ, tìm kiếm vector, hàng đợi, bộ đệm, lưu trữ đối tượng
Hạ tầng	Docker Compose, Prometheus, Grafana	Triển khai, khung giám sát

Backend, extension, bot và AI nằm chung một kho Git; frontend là kho riêng. Quy mô hiện tại: core-service khoảng 29 nghìn dòng TypeScript trong 64 module,

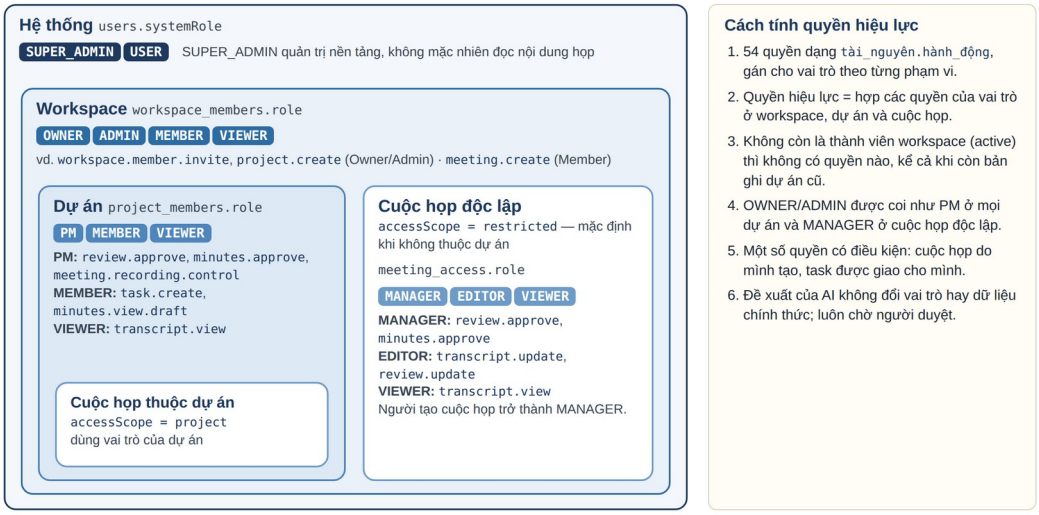
frontend khoảng 14,8 nghìn dòng với 29 trang, bot-worker khoảng 780 dòng, extension khoảng 700 dòng. Phần lớn module CRUD được sinh bằng công cụ `tools/generate-module.js` từ tệp đặc tả bảng, nên 43 bảng của thiết kế được hiện thực đồng nhất.

2.2. Xây dựng dịch vụ lõi core-service

2.2.1. Tổ chức mã nguồn và phân quyền

Core-service là nơi duy nhất sở hữu dữ liệu nghiệp vụ và quyết định quyền. Mỗi module theo khuôn cố định: thực thể, mô hình Sequelize, kho dữ liệu (repository), dịch vụ, bộ điều khiển và DTO kiểm tra đầu vào. `BaseService` và `SqlRepository` cung cấp sẵn thao tác CRUD, phân trang, xoá mềm; bộ điều khiển CRUD sinh từ `BaseControllerFactory`, mỗi module chỉ khai tuyến nào bật và cần quyền gì. Khung mẫu ban đầu có cả MongoDB; nhóm bỏ MongoDB, đưa toàn bộ dữ liệu về schema `core`. Thao tác nghiệp vụ nhiều bước (duyệt biên bản, duyệt đề xuất, mời bot) nằm ở các module riêng `workflow`, `meeting-processing`, `meeting-bot`, `me`.

Mô hình phân quyền vai trò kết hợp phạm vi [5, 17] (xem Hình 2.1) được hiện thực trong `PermissionService`, với danh mục 54 quyền dạng `<tài nguyên>.<hành động>` và ba bảng ánh xạ vai trò → quyền cho workspace, dự án và cuộc họp độc lập. Hàm `resolve` tính tập quyền hiệu lực theo thứ tự: chỉ thành viên `active` mới có quyền; cộng quyền vai trò workspace; nếu yêu cầu gắn với cuộc họp thì xác nhận cuộc họp thuộc đúng workspace rồi suy ra dự án; cộng quyền vai trò dự án; với cuộc họp `restricted` thì cộng quyền theo bảng `meeting_access`. OWNER và ADMIN được coi như PM ở mọi dự án của workspace; SUPER_ADMIN không được đi vòng qua, vì đó là quyền quản trị nền tảng chứ không phải quyền đọc nội dung khách hàng. Hai điều kiện theo bản ghi được xét riêng: thành viên được sửa cuộc họp *do chính mình tạo* khi vẫn còn quyền xem nó (`canOnMeeting`), người được giao task được đổi trạng thái task đó (`canOnTask`).

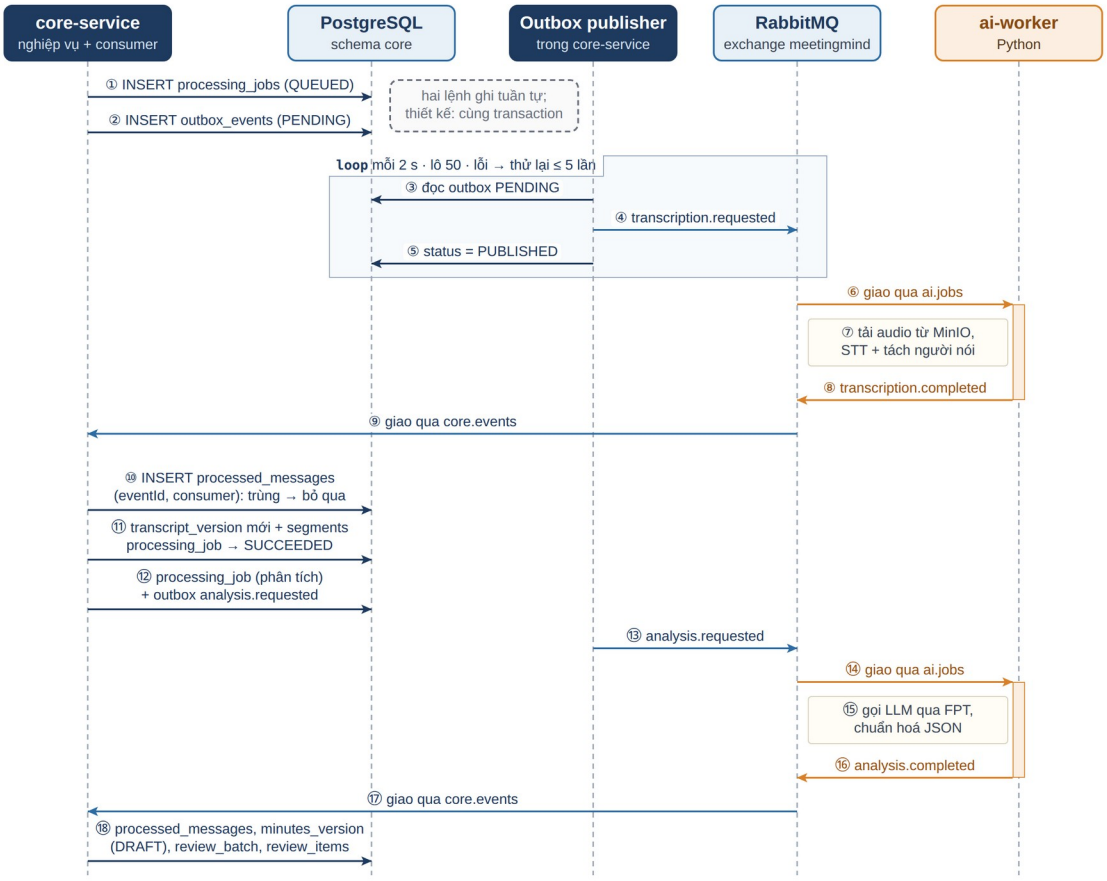


Hình 2.1. *Phân quyền theo vai trò kết hợp phạm vi tài nguyên*

`PermissionGuard` đọc quyền khai bằng `@RequirePermission` và xác định phạm vi từ header `x-workspace-id`, `x-project-id`, `x-meeting-id`, sau đó mới tới thân yêu cầu. Header là bắt buộc với yêu cầu multipart, vì multer phân tích thân yêu cầu *sau* khi guard chạy. Ban đầu guard xác thực và guard phân quyền gắn bằng hai decorator riêng; do decorator lớp áp dụng từ dưới lên, guard phân quyền đã chạy trước. Nhóm gộp cả hai vào decorator `Authorization()` để thứ tự luôn là xác thực JWT rồi mới phân quyền.

2.2.2. Outbox, publisher và chống xử lý lặp

Giao tiếp giữa core-service với ai-worker, bot-worker và integration-service đi qua RabbitMQ [14] theo mẫu transactional outbox [16] (xem Hình 2.2). Mọi sự kiện có chung phong bì (`eventId`, `eventType`, `correlationId`, `payload`…); hợp đồng viết một lần trong `contracts/events.json`, có bản dịch cho TypeScript và Python. ai-worker nghe hàng đợi `ai.jobs`; bot-worker nghe `bot.jobs` cho lệnh vào phòng và `bot.control` cho lệnh dừng — tách riêng vì phiên ghi âm kéo dài hàng giờ, dùng chung thì lệnh dừng phải xếp sau chính phiên cần dừng; integration-service nghe `integration.jobs.task-sync`.



Hình 2.2. *Sơ đồ tuần tự sự kiện giữa core-service và ai-worker*

`OutboxService.emit` không gửi thẳng mà ghi một dòng `outbox_events` trạng thái `pending`. `OutboxPublisherService` quét mỗi 2 giây, lấy tối đa 50 sự kiện cũ nhất, phát với `messageId` bằng `eventId` ở chế độ lưu bền rồi đánh dấu `published`; lỗi được đếm, quá 5 lần thì chuyển `failed` để người vận hành kiểm tra. Nhờ vậy khi RabbitMQ ngừng, lệnh ghi nghiệp vụ vẫn thành công và sự kiện chờ trong bảng [8]. Cần nói rõ hai hạn chế: `emit` nhận tham số `transaction` nhưng chưa nơi gọi nào truyền vào, nên job và sự kiện vẫn được ghi tuần tự, chưa cùng một giao dịch; hệ thống cũng chưa có hàng đợi thư chết (dead-letter queue) hay cơ chế tự chạy lại job lỗi. Đây là các việc cần hoàn thiện.

RabbitMQ chỉ bảo đảm giao tin ít nhất một lần (at-least-once), nên mọi consumer của Core phải chịu được tin lặp. `MessageDedupService.claim` ghi cặp `(messageId, consumer)` vào bảng `processed_messages` có chỉ mục duy nhất; lần thứ hai vi phạm ràng buộc và handler bỏ qua tin:

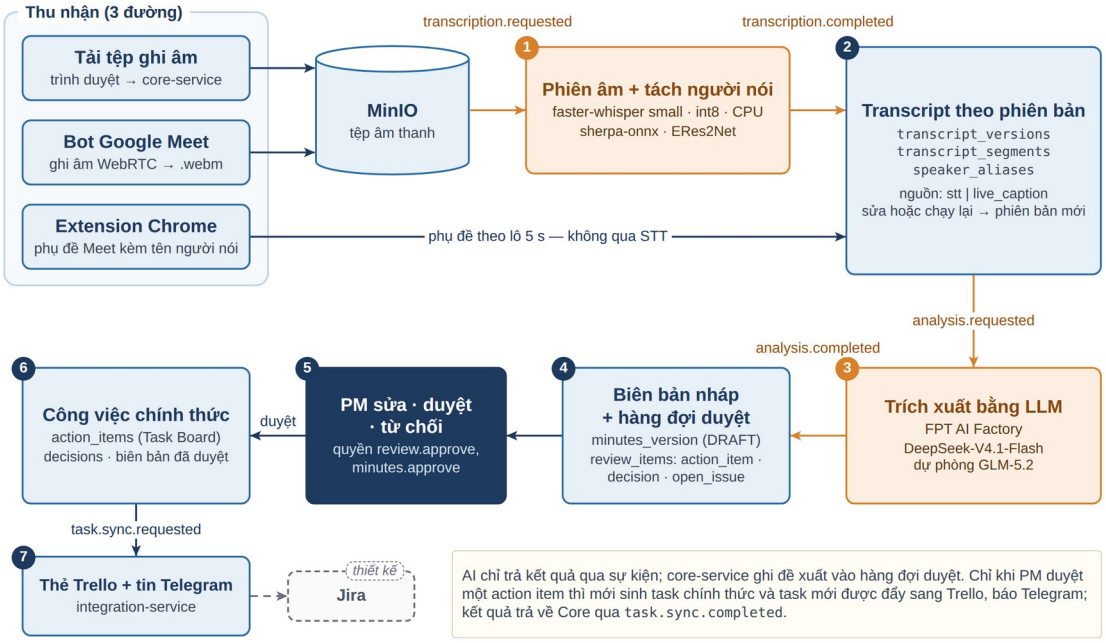
```
@RabbitSubscribe({
    exchange: MEETINGMIND_EXCHANGE,
    routingKey: EventType.TRANSCRIPTION_COMPLETED,
    queue: `${CORE_EVENTS_QUEUE}.transcription-completed`,
    queueOptions: { durable: true },
})
async onTranscriptionCompleted(envelope:
EventEnvelope<TranscriptionCompletedPayload>) {
    if (!(await this.dedup.claim(envelope.eventId, CONSUMER))) {
```

```
            return; // đã xử lý ở lần giao trước
        }
        await
this.meetingProcessingService.onTranscriptionCompleted(envelope.payload);
}
```

Ở chiều phát lệnh, mỗi `processing_jobs` mang khoá lũy đẳng (`idempotencyKey`), ví dụ `<meetingId>:transcribe:<objectKey>`; gọi lại với cùng đầu vào trả về job đã có. Nguyên tắc này áp dụng nhất quán: mời bot khi đang có phiên hoạt động thì trả về phiên đó, duyệt lại một đề xuất đã duyệt không sinh thêm task, sự kiện trạng thái bot đến trễ không làm lùi phiên đã kết thúc.

2.2.3. Điều phối luồng xử lý cuộc họp

`MeetingProcessingService` hiện thực luồng ở Hình 2.3 theo nguyên tắc Core ra lệnh và ghi kết quả, AI chỉ tính toán rồi trả về. Khi nhận transcript, dịch vụ tạo `transcript_versions` mới (không ghi đè bản cũ), ghi từng đoạn kèm nhãn người nói, mốc thời gian, độ tin cậy, ngôn ngữ và cờ nói chồng, rồi cập nhật ngôn ngữ cuộc họp theo ngôn ngữ nói nhiều nhất. Lệnh phân tích gửi kèm toàn bộ transcript, vì ai-worker không được đọc schema `core`. Khi nhận kết quả phân tích, dịch vụ tạo biên bản nháp, một lô duyệt và các đề xuất chờ duyệt.

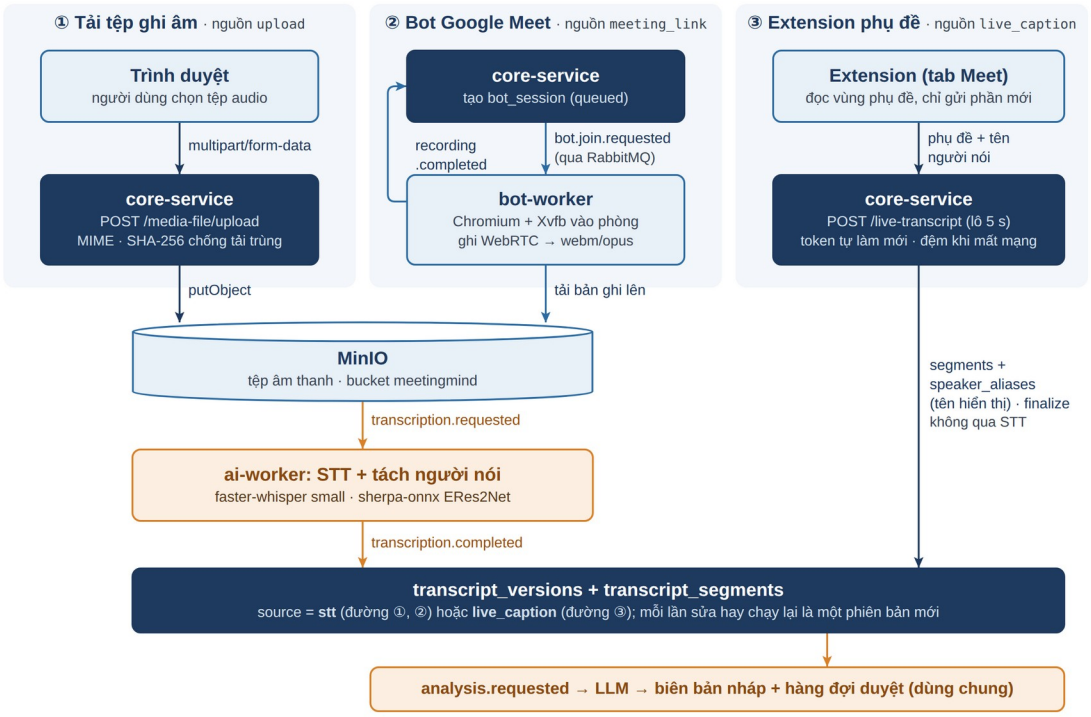


Hình 2.3. *Luồng xử lý từ âm thanh đến công việc*

2.3. Ba đường thu nhận nội dung cuộc họp

Hệ thống hỗ trợ ba cách đưa nội dung vào, ứng với ba bối cảnh: tải lên tệp ghi âm có sẵn (họp trực tiếp), mời bot ghi âm cuộc họp Google Meet (cách của Otter [12], Fireflies [3]), và extension đọc phụ đề do Meet sinh ra (cách của Tactiq [20]). Như

Hình 2.4 minh hoạ, hai đường đầu đi qua nhận dạng tiếng nói, đường thứ ba thì không; cả ba hội tụ tại `transcript_versions`, từ đó phần phân tích và duyệt dùng chung.



Hình 2.4. *Ba đường thu nhận nội dung cuộc họp hội tụ tại transcript_versions*

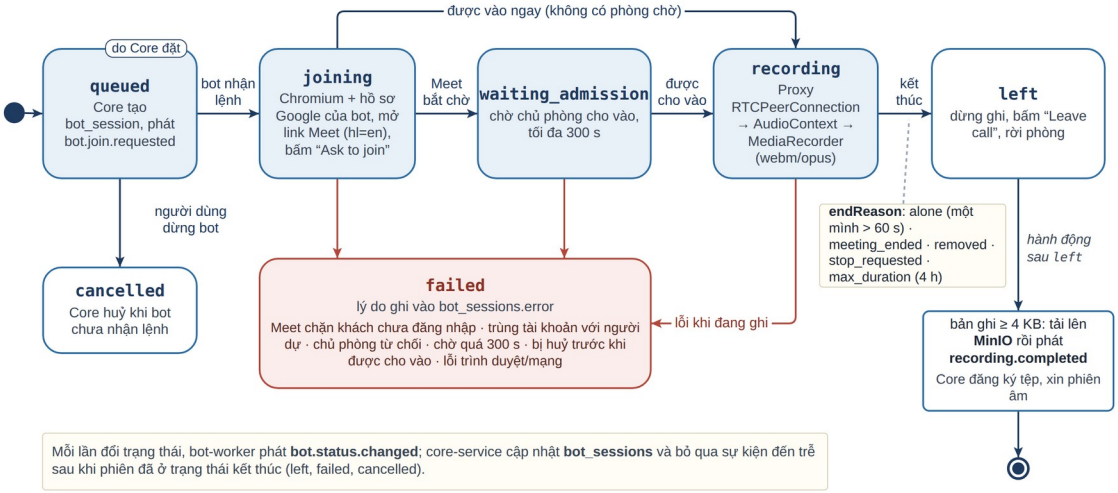
2.3.1. Tải tệp ghi âm lên MinIO

Bảng `files` chỉ lưu `bucket` và `objectKey`; khung mẫu cũ lưu tệp base64 trong cơ sở dữ liệu, không dùng được cho bản ghi một giờ. `MediaFileUploadService` kiểm tra kiểu MIME, tính băm SHA-256 và nếu workspace đã có tệp trùng băm thì dùng lại bản ghi cũ thay vì tạo object trùng. Tệp mới được đặt khoá `<workspaceId>/<meetingId>/<uuid>.<đuôi>`, chuyển trạng thái `uploading` → `ready` (hoặc `failed`). Để nghe lại, Core cấp URL ký có hạn 300 giây sau khi kiểm quyền `audio.listen`. Khi chạy thật, nhóm phát hiện chữ ký URL gắn chặt với tên máy: ký bằng tên nội bộ `minio` thì trình duyệt nhận lỗi 403; giải pháp là thêm `MINIO_PUBLIC_ENDPOINT` và một client riêng chỉ để ký. Với bot, Core chỉ đăng ký siêu dữ liệu của bản ghi bot tự đẩy lên MinIO, không làm trung gian cho tệp lớn.

2.3.2. Bot tham gia và ghi âm Google Meet

Ở core-service, `MeetingBotService` kiểm tra quyền `meeting.recording.control` và đường dẫn đúng dạng `https://meet.google.com/...`, tạo `bot_sessions` rồi phát `bot.join.requested`; khi nhận `recording.completed`, Core đăng ký tệp và gọi đúng hàm bắt đầu phiên âm của đường tải tệp. Bot-worker mở hai kênh RabbitMQ: kênh việc nhận một phiên mỗi lúc (một hồ sơ Chromium không mở được hai lần) và kênh điều khiển để nhận lệnh

dừng. Lệnh vào phòng được xác nhận (ack) ngay, vì phiên dài hơn `consumer_timeout` 30 phút sẽ bị broker đóng kênh.



Hình 2.5. *Trạng thái một phiên bot và các lý do kết thúc*

Vòng đời một phiên ở Hình 2.5: `queued` khi Core tạo, `joining` khi mở trang, `waiting_admission` khi chờ chủ phòng duyệt, `recording` khi đã vào và bắt đầu ghi. Phiên kết thúc ở `left` khi chỉ còn một mình quá 60 giây, cuộc họp kết thúc, bị mời ra, có lệnh dừng hoặc vượt 4 giờ; thất bại ở `failed` khi Meet từ chối khách, chủ phòng từ chối hay chờ duyệt quá 300 giây; bị dừng khi còn `queued` thì thành `cancelled`. Tải lên không phải một trạng thái riêng: sau `left`, bản ghi dưới 4 KB (không ai nói) bị bỏ, còn lại được bot tải lên MinIO rồi phát `recording.completed`.

Bot dùng Playwright [10] điều khiển Chromium có giao diện trên màn hình ảo Xvfb, vì Meet hay từ chối trình duyệt không giao diện. Bot không bật micro, camera; âm thanh được ghi ngay trong trang bằng script chèn trước mọi script của Meet. Script bọc hàm tạo `RTCPeerConnection` [21] để bắt mọi track âm thanh từ xa, trộn vào một `AudioContext` rồi ghi bằng `MediaRecorder` dạng webm/opus 64 kbps, mỗi khúc 10 giây được đẩy về Python theo đúng thứ tự.

```
const Original = window.RTCPeerConnection;
window.RTCPeerConnection = new Proxy(Original, {
    construct(target, args) {
        const pc = Reflect.construct(target, args);
        // Mỗi track âm thanh từ xa được trộn vào AudioContext chung.
        pc.addEventListener("track", (event) => attach(event.track));
        return pc;
    },
});
```

Phần tốn công nhất là các tình huống thực tế của Google Meet:

– **Google chặn khách ẩn danh dùng trình duyệt tự động** ("You can't join this video call"), nên bot bắt buộc dùng tài khoản Google riêng. Người vận hành đăng

nhập một lần qua noVNC bằng `login.sh` (Chromium thường, vì trang đăng nhập Google cũng từ chối trình duyệt tự động); hồ sơ kèm cookie lưu ở volume `bot-profile`. Nếu hồ sơ bot trùng tài khoản với người trong phòng, Meet đề nghị "Switch here" — bấm vào sẽ đẩy người thật ra — nên bot nhận ra nút này và dừng với thông báo lỗi.

- **Giao diện Meet theo ngôn ngữ tài khoản**, không theo locale trình duyệt, nên với tài khoản tiếng Việt nút hiện "Yêu cầu tham gia". Nhóm ép giao diện tiếng Anh bằng tham số `hl=en` trên đường dẫn, giữ nhãn tiếng Việt làm dự phòng.
- **Nhãn nút dài, dễ khớp nhầm.** Nút thật tên "Ask to join without microphone & camera", còn so khớp lỏng "Join" lại trúng "Other ways to join"; nhóm chuyển sang so khớp tiền tố (`^Ask to join`).
- **Phòng chờ kiểu mới** cũng có nút "Leave call", nên bot chỉ coi là đã vào khi dòng "Asking to be let in" biến mất.
- **Thiếu thiết bị âm thanh.** Trong container không có loa, âm thanh WebRTC của người khác không chảy và bot chỉ ghi được khoảng lặng; nhóm khởi động PulseAudio với loa ảo (null sink). Ngoài ra, do một lỗi đã biết của Chromium, mỗi track từ xa phải gắn vào một thẻ `<audio>` tắt tiếng thì mới vào được Web Audio.
- **Hồ sơ trình duyệt của bot phải sạch.** Hồ sơ lưu phiên đăng nhập của bot không được kéo extension, tab hay lịch sử đồng bộ từ thiết bị khác, vì các thành phần lạ chèn vào trang có thể làm thay đổi giao diện Meet mà bot đang dò theo. Chromium của bot được chạy với `--disable-sync` cả khi đăng nhập lẫn khi vào phòng.
- **Hạ tầng:** `xvfb-run` treo khi là tiến trình số 1 nên Xvfb được khởi động trực tiếp; `/dev/shm` mặc định 64 MB làm tab sập nên container được cấp 1 GB.

Bot không vượt rào kiểm soát của Meet: hiện đúng tên, phải được chủ phòng duyệt và dừng ngay khi bị từ chối.

2.3.3. Tiện ích mở rộng đọc phụ đề

Extension Chrome theo Manifest V3 [4] không có bot và không ghi âm, chỉ đọc phụ đề Meet đã hiển thị trên màn hình của chính người dùng, nên có sẵn tên người nói. Người dùng bấm "Bắt đầu" trong popup; Core tìm cuộc họp theo đường dẫn phòng, chưa có thì tạo mới; khi kết thúc, Core chốt phiên bản transcript và gọi lại đúng bước phân tích của hai đường kia. Tên hiển thị trên Meet chỉ được lưu ở mức `suggested`, vì đó không phải bằng chứng danh tính.

`content.js` quét DOM mỗi 400 ms thay vì gắn `MutationObserver` cố định, vì Meet gỡ và dựng lại vùng phụ đề sau mỗi quãng im lặng. Vùng phụ đề được tìm theo thuộc tính trợ năng `[role="region"]` có `aria-label` chứa "Captions" hoặc "Phụ đề", không dựa vào tên lớp CSS bị làm rối; khối con có từ hai vùng chữ trở lên là một lượt nói, vùng đầu là tên. Thử trên phòng Meet thật ngày 29/10/2026 cho thấy ba lỗi. Thứ nhất là **lặp chồng**: Meet giữ một khối cho cùng người nói và nối câu mới vào sau, trong khi mã cũ gửi cả khối mỗi khi chữ đứng yên, nên transcript thành "A", "A B", "A B C". Mỗi khối giờ nhớ phần đã gửi và chỉ gửi phần mới, xử lý cả khi Meet cắt dòng đầu hay sửa từ đã gửi:

```
function unsentPart(sent, live) {
    if (live.startsWith(sent)) return live.slice(sent.length).trim(); // nói
tiếp
    const tail = sent.slice(-TAIL_LEN);                                // 24
ký tự cuối
    const at = tail.length === TAIL_LEN ? live.lastIndexOf(tail) : -1;
    if (at >= 0) return live.slice(at + TAIL_LEN).trim();              // Meet
cắt dòng đầu
    // Meet sửa từ đã gửi: gửi lại từ đầu từ bị sửa, không gửi nửa từ.
    const p = commonPrefixLength(sent, live);
    const atBoundary = p === 0 || /\s/.test(live[p - 1]) || /\s/.test(live[p]
?? " ");
    const from = atBoundary ? p : live.lastIndexOf(" ", p) + 1;
    return live.slice(from).replace(/^[\s.,!?…]+/, "").trim();
}
```

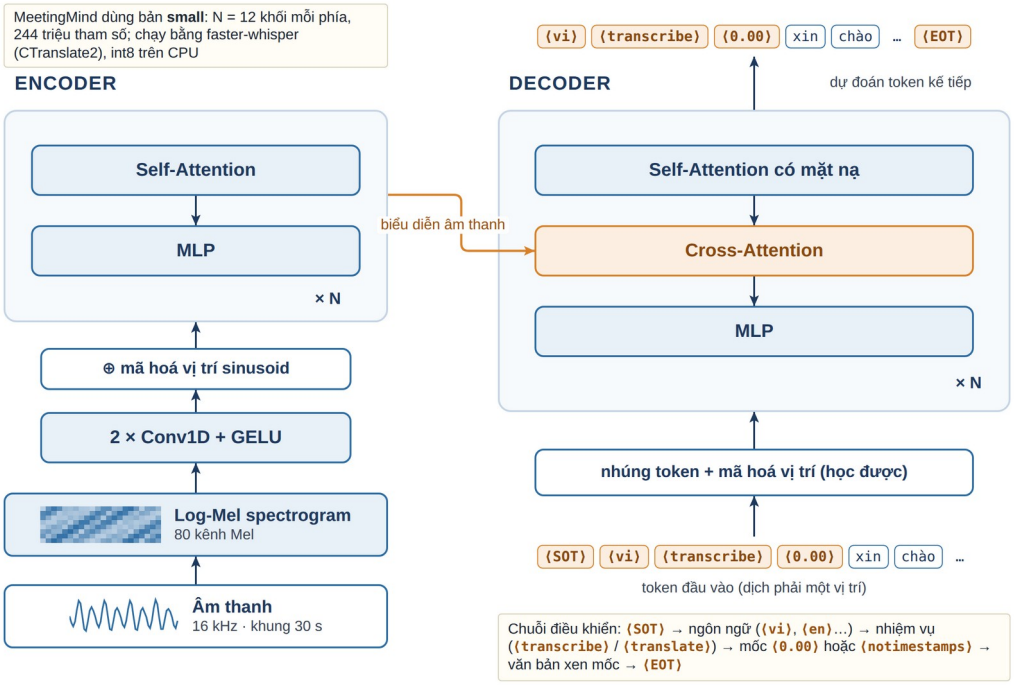
Thứ hai là **bắt nhầm rác lúc im lặng**: khi vùng phụ đề rỗng, mã cũ rơi xuống bộ chọn dự phòng và đọc nhầm menu cài đặt thành phụ đề; các bộ chọn dự phòng đã bị bỏ. Thứ ba là **khai báo trùng biến**: các content script của cùng extension chung một phạm vi toàn cục, nên `unsentPart` khai ở cấp ngoài cùng của `caption-text.js` đụng `const { unsentPart }` trong `content.js`, làm `content.js` lỗi cú pháp và không chạy, trong khi popup chỉ báo "Không gọi được tab Meet". Nhóm bọc `caption-text.js` trong hàm tự gọi (IIFE) và viết kiểm thử nạp các content script theo thứ tự manifest vào chung một ngữ cảnh.

Service worker MV3 bị Chrome tắt sau khoảng 30 giây không có sự kiện, làm mất biến trong bộ nhớ và `setInterval`, nên phụ đề từng nằm mãi trong bộ đệm. Bộ đệm được chuyển vào `chrome.storage.local`, việc gửi theo lô 5 giây được hẹn bằng `setTimeout` mỗi khi có đoạn mới; gửi lỗi thì lô trả về bộ đệm, gặp mã 401 thì làm mới token rồi gửi lại. Hạn chế: người dùng phải bật phụ đề trong Meet và extension phụ thuộc cấu trúc DOM của Meet.

2.4. Mô-đun nhận dạng tiếng nói

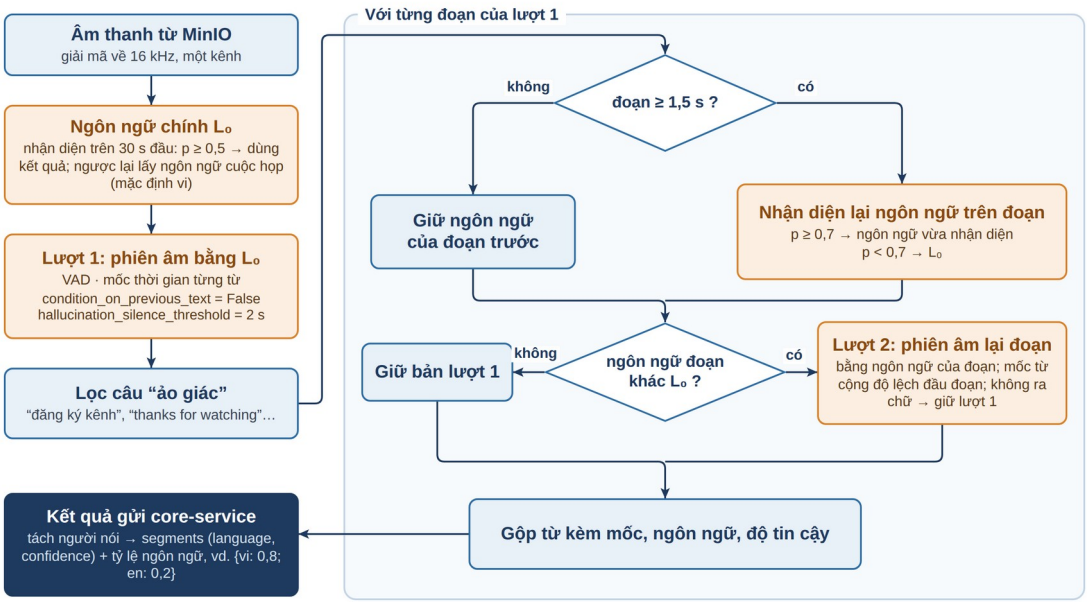
Mô-đun nhận dạng tiếng nói (Speech-to-Text, STT) dùng faster-whisper 1.2.1 [19], bản cài đặt Whisper [15] trên CTranslate2 (xem Hình 2.6), mặc định mô hình

`small`, chạy CPU lượng tử hoá int8, beam search độ rộng 5; mô hình nạp một lần mỗi tiến trình. Phiên âm chạy trong luồng phụ (`asyncio.to_thread`), vì chặn vòng lặp sự kiện sẽ làm lỡ heartbeat và RabbitMQ cắt kết nối giữa chừng.



Hình 2.6. *Kiến trúc mô hình Whisper với chuỗi token đặc biệt điều khiển ngôn ngữ, nhiệm vụ và mốc thời gian*

Whisper chỉ nhận diện ngôn ngữ một lần trên 30 giây đầu; cuộc họp nói lẫn Việt – Anh thì câu tiếng Anh bị giải mã như tiếng Việt thành chữ vô nghĩa hoặc bị dịch. Nhóm thiết kế phiên âm hai lượt (Hình 2.7). Lượt một chọn ngôn ngữ chính (xác suất dưới 0,5 thì theo ngôn ngữ cuộc họp do Core gửi) và phiên âm có lọc khoảng lặng (VAD). Lượt hai nhận diện lại ngôn ngữ trên từng đoạn từ 1,5 giây; nếu khác ngôn ngữ chính với xác suất từ 0,7 thì phiên âm lại đoạn đó bằng ngôn ngữ của nó và cộng bù mốc thời gian. Đoạn ngắn hơn giữ ngôn ngữ đoạn trước, vì người nói hiếm khi đổi thứ tiếng giữa một câu ngắn.



Hình 2.7. *Phiên âm hai lượt theo ngôn ngữ của từng đoạn*

```
DECODE = {"condition_on_previous_text": False, "word_timestamps": True,
          "hallucination_silence_threshold": 2.0}

def choose_segment_language(base, previous, duration_s, guess, settings):
    if duration_s < settings.min_detect_seconds or guess is None:
        return previous   # đoạn dưới 1,5 s: giữ ngôn ngữ đoạn trước
    language, probability = guess
    if probability >= settings.segment_min_prob:
        return language   # nhận diện lại đủ chắc (>= 0,7)
    return base           # không chắc: về ngôn ngữ chính
```

Tham số giải mã nhằm hạn chế "ảo giác" (sinh câu không có trong âm thanh) [6]: `condition_on_previous_text=False` tránh vòng lặp lặp câu; `hallucination_silence_threshold=2.0` bỏ đoạn bịa giữa khoảng lặng dài, cần `word_timestamps=True` — vốn cũng là đầu vào của bước tách người nói. Một biểu thức chính quy lọc thêm các câu Whisper hay bịa do học từ phụ đề video như "đăng ký kênh", "thanks for watching". Mỗi đoạn mang độ tin cậy và ngôn ngữ riêng; bản ghi kèm tỷ lệ thời lượng theo ngôn ngữ.

Hai lỗi phát sinh khi chạy thật đáng ghi lại. PyAV bản 19 bỏ tham số `metadata_errors` mà faster-whisper 1.2.1 vẫn truyền vào hàm mở tệp, khiến mọi lần giải mã lỗi; nhóm ghim `av==18.1.0`. Mốc thời gian của faster-whisper là số kiểu NumPy, nên cờ nói chồng tính từ chúng có kiểu `numpy.bool` không chuyển được sang JSON, kết quả không gửi về được và job ở Core kẹt ở trạng thái chờ; nhóm thêm hàm chuyển kiểu khi tuần tự hoá và đưa lệnh gửi vào khối bắt lỗi để mọi lỗi đều thành `transcription.failed`. Độ chính xác được đánh giá ở chương Thực nghiệm và đánh giá (Hình 3.1).

2.5. Mô-đun tách người nói

Với cuộc họp trực tiếp ghi bằng một micro, mô-đun tách người nói (speaker diarization) [13] trả lời câu hỏi "ai nói khi nào". Nhóm thử trước mô hình phân đoạn pyannote [1], nhưng trên dữ liệu thử mô hình coi cả cửa sổ 10 giây là một người và gộp nhầm lời hai người, trong khi vector đặc trưng giọng tách người rất rõ (độ giống cosine khoảng 0,6 giữa hai khúc cùng người, khoảng 0,2 giữa hai người khác nhau). Vì vậy nhóm chuyển sang gom cụm trực tiếp các khúc lời (Hình 2.8).



Hình 2.8. *Quy trình tách người nói bằng gom cụm vector đặc trưng giọng*

Lời Whisper được cắt thành khúc khoảng 1,5 giây theo mốc từ, cắt thêm khi gặp khoảng lặng trên 0,4 giây hoặc sang đoạn Whisper mới. Mỗi khúc được tính vector đặc trưng giọng bằng mô hình ERes2Net của bộ 3D-Speaker [2] qua sherpa-onnx [7] trên CPU — ý tưởng biểu diễn người nói bằng vector nhúng cố định độ dài kế thừa từ x-vector [18]. Các vector được gom cụm phân cấp theo liên kết trung bình (average linkage): liên tục ghép hai cụm có độ tương đồng cosine cao nhất, chừng nào độ tương đồng đó còn từ 0,3 trở lên, nên số người được suy ra chứ không khai trước. Thu âm ồn hay sinh "người nói" ảo chỉ vài giây, nên cụm có tổng thời gian nói dưới 2 giây được gộp vào cụm lớn giống nhất; đánh đổi là người chỉ nói một câu "ok" có thể bị gộp vào người khác.

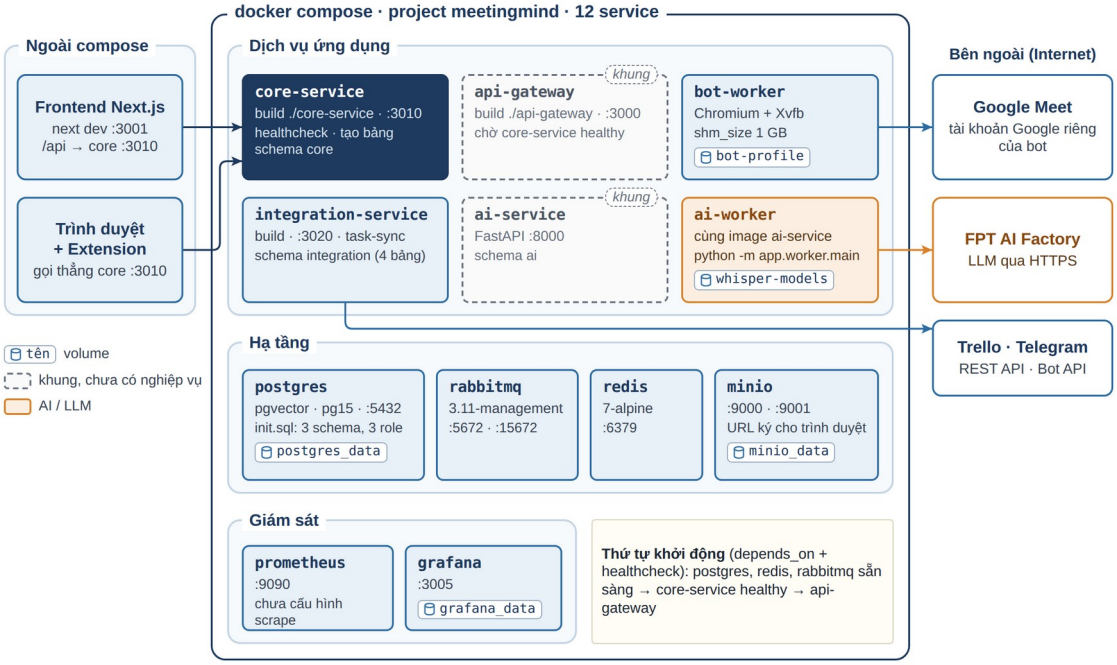
Để đánh dấu nói chồng, mô-đun xét độ giống của mỗi khúc với tâm cụm *đứng thứ hai*: khi hai người nói chồng, giọng át vẫn trội nhưng độ giống với người kia nhích lên. Mức "bình thường" khác nhau theo bản ghi, nên ngưỡng được tính thích nghi bằng trung vị cộng hai lần độ lệch tuyệt đối trung vị (MAD), với sàn 0,3. Từ các khúc đã gán người, mô-đun ghép ngược với lời Whisper: mỗi từ thuộc lượt nói phủ nó nhiều

nhất, một đoạn Whisper trùm hai người được cắt đôi đúng chỗ đổi người, nhãn "Người nói 1, 2…" đặt theo thứ tự lên tiếng, và đoạn có phần chồng lấn từ 0,3 giây (hoặc từ 30% thời lượng) được gắn cờ `isOverlapping`. Sau khi thực nghiệm cho thấy cơ chế này báo nhầm nhiều (chương Thực nghiệm và đánh giá), cờ được tắt mặc định và chỉ bật khi đặt biến `AI_DIARIZATION_OVERLAP=1` để nghiên cứu tiếp.

Tách người nói là phần bổ sung, không được làm hỏng phiên âm: toàn bộ bước này nằm trong khối bắt lỗi, lỗi chỉ được ghi log và mọi đoạn nhận chung nhãn "Người nói 1"; biến `AI_DIARIZATION=0` tắt hẳn. Giới hạn bản chất là với một micro, lời người nói chồng thường bị Whisper bỏ mất, nên cờ nói chồng chỉ là gợi ý "nên nghe lại"; nhãn "Người nói N" cũng không phải danh tính, người dùng gán tên ở màn hình transcript.

2.6. Triển khai và vận hành

Backend được mô tả trong một tệp `docker-compose.yml` gồm 12 dịch vụ (Hình 2.9): PostgreSQL 15, Redis 7, RabbitMQ 3.11, MinIO, api-gateway, core-service, integration-service, ai-service, ai-worker, bot-worker, Prometheus và Grafana; frontend chạy riêng. ai-worker dùng chung ảnh với ai-service nhưng chạy tiến trình nhận việc từ RabbitMQ, còn ai-service chạy máy chủ FastAPI cổng 8000 phục vụ hỏi đáp cho core-service. Thứ tự khởi động ràng buộc bằng kiểm tra sức khoẻ; chỉ dịch vụ sở hữu schema được bật tự sinh bảng, tránh nhiều tiến trình cùng sửa bảng lúc khởi động. Tệp `init.sql` tạo ba schema `core`, `integration`, `ai` cùng ba tài khoản riêng và thu hồi quyền chéo giữa các schema.



Hình 2.9. *Triển khai hệ thống bằng Docker Compose: các container và volume*

Dữ liệu bền giữ trong năm volume: `postgres_data`, `minio_data`, `grafana_data`, `whisper-models` (mô hình Whisper và ERes2Net, tải lần đầu) và `bot-profile` (hồ sơ Chromium đã đăng nhập tài khoản bot). Cấu hình đi qua biến môi trường: `AI_STT`, `AI_WHISPER_MODEL`, `AI_DIARIZATION` cho phiên âm; `AI_LLM`, `AI_LLM_MODEL`, `AI_LLM_FALLBACK_MODEL` cho phân tích; `BOT_ADMISSION_TIMEOUT_S`, `BOT_ALONE_TIMEOUT_S` cho bot; `TRELLO_API_KEY`, `TRELLO_TOKEN`, `TRELLO_BOARD_ID` (hoặc `TRELLO_LIST_ID`), `TELEGRAM_BOT_TOKEN`, `TELEGRAM_CHAT_ID` cho đồng bộ; `AI_EMBEDDING_MODEL`, `AI_RERANK_MODEL`, `AI_QA_MODEL`, `AI_QA_FALLBACK_MODEL`, `AI_INTERNAL_TOKEN` cho hỏi đáp. Tệp `.env` chứa khoá LLM, khoá nội bộ giữa Core và ai-service, khoá Trello, token Telegram và khoá ký gói extension (`*.pem`) đều bị `.gitignore` loại; các mật khẩu mặc định trong `docker-compose.yml` chỉ dành cho môi trường phát triển.

Về giám sát, Prometheus và Grafana đã được khai báo nhưng chưa có cấu hình thu thập số liệu, các dịch vụ cũng chưa xuất điểm cuối số liệu, nên giám sát mới ở mức khung. Khi vận hành và gỡ lỗi, nhóm dựa vào log của từng dịch vụ, trang quản trị RabbitMQ để xem hàng đợi, giao diện MinIO để kiểm tra object, trạng thái trong `processing_jobs`, `bot_sessions`, và ảnh chụp từng bước của bot.

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ

3.1. Môi trường và bộ dữ liệu thực nghiệm

3.1.1. Môi trường

Toàn bộ thực nghiệm chạy trên một máy tính xách tay cá nhân, không dùng GPU, để phản ánh điều kiện triển khai thực tế của một nhóm nhỏ. Mô-đun AI chạy trong đúng container ai-worker của hệ thống, nên kết quả phản ánh chính mã nguồn được triển khai. Cấu hình được tóm tắt trong Bảng 3.1.

Bảng 3.1. *Môi trường thực nghiệm*

Hạng mục	Cấu hình
CPU	Intel Core i7-10750H (6 nhân, 12 luồng, 2,6 GHz)
Bộ nhớ	23 GB RAM, không có GPU
Hệ điều hành, đóng gói	Linux, Docker Compose (container ai-worker của hệ thống)
Nhận dạng tiếng nói	faster-whisper 1.2.1, mô hình small và medium, int8 trên CPU, beam size 5
Tách người nói	sherpa-onnx 2.0.1, ERes2Net (3D-Speaker), ngưỡng tương đồng 0,3
LLM	FPT AI Factory Serverless Inference, nhiệt độ 0,2
Hỏi đáp tri thức	Vietnamese_Embedding, bge-reranker-v2-m3, GLM-5.2 (FPT AI Factory); pgvector 0.8.6
Giọng tổng hợp dựng dữ liệu	FPT.AI-VITs (các giọng banmai, leminh, giahuy, ngoclam, kimngan, huyphong, minhquan)

3.1.2. Kịch bản cuộc họp có đáp án

Do không có sẵn bộ dữ liệu cuộc họp tiếng Việt công khai kèm nhãn công việc, nhóm tự xây dựng một kịch bản họp lập kế hoạch sprint của một nhóm phát triển ứng dụng đặt lịch phòng khám, gồm 30 lượt lời của 4 người (Lan – trưởng nhóm, Nam, Hùng và một thành viên chỉ xuất hiện dưới nhãn "Người nói 4" nhưng được gọi tên là Phương ở cuối buổi). Kịch bản được thiết kế có chủ đích để chứa các tình huống mà một trợ lý biên bản dễ làm sai:

- **Đề xuất chưa được chấp nhận:** "Hay là mình dùng luôn Firebase cho OTP?" không phải quyết định, mà dẫn tới một việc so sánh giá và một vấn đề còn mở.
- **Đính chính thời hạn:** hạn tích hợp thanh toán được nói là ngày 15 tháng 11 rồi sửa ngay thành ngày 18.
- **Đổi người thực hiện:** việc nhắc lịch "ban đầu định giao Nam" nhưng cuối cùng giao cho Hùng.
- **Việc thiếu thời hạn:** nhiều việc không nêu hạn, hệ thống không được tự điền.

- **Chuyện ngoài lề:** hai lượt lời bàn về bóng đá không được biến thành công việc.
- **Người nói chưa có tên:** việc viết tài liệu API của "Người nói 4" phải được gán cho Phương nhờ câu tổng kết.
- **Thuật ngữ tiếng Anh:** sprint, API, OTP, Firebase, eSMS, PostgreSQL, MongoDB, VNPay, MoMo, callback, Zalo, SMS, Swagger.

Đáp án gồm 8 công việc (kèm người thực hiện và thời hạn đúng), 2 quyết định (chốt PostgreSQL, chưa làm MoMo trong sprint này) và 3 vấn đề còn mở (chọn Firebase hay eSMS, gửi nhắc lịch qua Zalo hay SMS, ngân sách máy chủ).

3.1.3. Dựng âm thanh bằng giọng tổng hợp

Từ kịch bản, mỗi lượt lời được tổng hợp thành tiếng nói bằng dịch vụ FPT.AI-VITs, mỗi người dùng một giọng khác nhau (hai giọng nam, hai giọng nữ). Các câu được ghép với khoảng nghỉ ngẫu nhiên 0,3–0,9 giây thành một bản ghi dài 151 giây, với mốc thời gian và người nói chuẩn của từng câu được lưu lại. Từ bản sạch, nhóm tạo thêm các điều kiện khó hơn như trong Bảng 3.2. Bản ghi tham chiếu có 443 tiếng (âm tiết) và 24 lần xuất hiện thuật ngữ tiếng Anh.

Bảng 3.2. *Các điều kiện âm thanh thử nghiệm*

Điều kiện	Cách tạo	Mục đích
Sạch	Ghép các câu tổng hợp, không thêm nhiễu	Mức chuẩn
Nhiễu trắng 20 / 10 / 5 dB	Cộng nhiễu Gauss theo tỷ số tín hiệu trên nhiễu (SNR)	Micro kém, quạt, điều hoà
Đông người 10 dB	Cộng tiếng nói chuyện nền của 3 giọng khác (6 câu không liên quan, lặp lại chồng nhau)	Phòng họp mở, nhiều người nói xung quanh
Nói chồng + đoạn nhiễu	4 chỗ người sau chen vào 1,2 giây cuối câu người trước; chèn 8 giây chỉ có nhiễu nền; nhiễu nhẹ 25 dB	Kiểm tra phát hiện nói chồng và hiện tượng ảo giác

Cách dựng dữ liệu bằng giọng tổng hợp cho phép kiểm soát hoàn toàn đáp án và điều kiện nhiễu, nhưng có hạn chế là giọng tổng hợp đều và rõ hơn giọng nói tự nhiên, đồng thời đọc thuật ngữ tiếng Anh theo cách phát âm tiếng Việt. Hạn chế này được bàn ở cuối chương.

3.2. Độ đo đánh giá

Tỷ lệ lỗi từ (WER) và tỷ lệ lỗi ký tự (CER). WER được tính bằng khoảng cách chỉnh sửa giữa chuỗi tiếng của bản nhận dạng và bản chuẩn [9], sau khi đưa về chữ thường và bỏ dấu câu. Do Whisper viết số bằng chữ số ("thứ 4", "12") còn bản chuẩn viết bằng chữ ("thứ Tư", "mười hai"), nhóm báo cáo thêm WER sau khi chuẩn hoá

cách viết số (đọc số 0–99 thành chữ, thứ trong tuần thành chữ). Đây là bước chuẩn hoá văn bản thông thường khi đánh giá nhận dạng tiếng nói, vì hai cách viết mang cùng nội dung. Trong chương này, "WER" được hiểu là WER đã chuẩn hoá số, trừ khi ghi khác.

Độ đúng thuật ngữ. Số lần thuật ngữ trong danh sách ở trên được nhận dạng đúng nguyên dạng, trên tổng 24 lần xuất hiện.

Độ đúng người nói. Với mỗi đoạn transcript đầu ra, tính thời lượng trùng với từng câu chuẩn; tìm cách ghép một-một tốt nhất giữa nhãn máy ("Người nói 1, 2…") và người thật; độ đúng là tỷ lệ thời lượng được gán đúng người trên tổng thời lượng trùng. Độ đo này tương ứng với phần "nhầm người" của DER, không tính phần bỏ sót hay báo thừa lời nói (vốn do Whisper quyết định).

Phát hiện nói chồng. Số vùng nói chồng chuẩn có ít nhất một đoạn được gắn cờ trùng lên (độ nhạy), và số đoạn bị gắn cờ trên các bản ghi không có nói chồng (báo nhầm).

Điểm trích xuất biên bản. Bộ chấm tự động gồm 15 tiêu chí: 8 công việc với người thực hiện và thời hạn đúng (kể cả việc đã đổi người, hạn đã đính chính), 1 tiêu chí gán đúng tên Phương cho "Người nói 4", 2 quyết định, 3 vấn đề còn mở và tóm tắt viết bằng tiếng Việt. Ngoài ra có 7 "bẫy", mỗi bẫy dính bị trừ 1 điểm: giữ hạn 15/11 chưa đính chính, giao nhắc lịch cho Nam, biến MoMo hoặc chuyện bóng đá thành công việc, coi đề xuất Firebase hoặc nâng gói máy chủ là quyết định, tự bịa thời hạn. Bộ chấm khớp theo từ khoá nên khá nghiêm: một công việc đúng nhưng ghi "thứ 4" thay vì "thứ Tư" vẫn bị coi là thiếu; với đầu ra từ transcript do máy nhận dạng, nhóm báo cáo cả điểm nghiêm ngặt và điểm sau khi chuẩn hoá cách viết thứ trong tuần.

Thời gian và chi phí. Thời gian thực thi từng bước và hệ số thời gian thực (RTF, thời gian xử lý chia thời lượng âm thanh); chi phí LLM tính theo số token và bảng giá của nhà cung cấp.

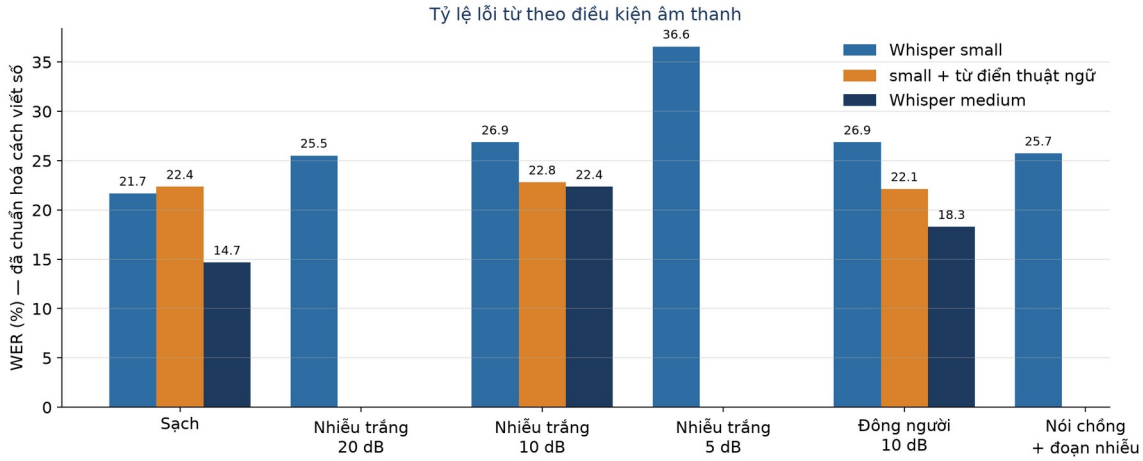
3.3. Kết quả nhận dạng tiếng nói

Bảng 3.3 và Hình 3.1 trình bày kết quả nhận dạng của Whisper small (cấu hình mặc định của hệ thống), Whisper small có từ điển thuật ngữ, và Whisper medium.

Bảng 3.3. *Kết quả nhận dạng tiếng nói theo điều kiện âm thanh*

Điều kiện	Cấu hình	WER thô (%)	WER chuẩn hoá số (%)	CER (%)	Thuật ngữ đúng
Sạch	small	26,6	21,7	15,8	2/24
Sạch	small + từ điển	26,9	22,4	18,8	11/24
Sạch	medium	18,7	14,7	10,9	7/24

Điều kiện	Cấu hình	WER thô (%)	WER chuẩn hoá số (%)	CER (%)	Thuật ngữ đúng
Nhiễu trắng 20 dB	small	30,0	25,5	16,8	3/24
Nhiễu trắng 10 dB	small	31,1	26,9	19,7	7/24
Nhiễu trắng 10 dB	small + từ điển	27,1	22,8	17,9	16/24
Nhiễu trắng 10 dB	medium	26,0	22,4	14,2	10/24
Nhiễu trắng 5 dB	small	40,9	36,6	25,5	6/24
Đông người 10 dB	small	31,6	26,9	18,5	3/24
Đông người 10 dB	small + từ điển	26,6	22,1	16,2	12/24
Đông người 10 dB	medium	22,6	18,3	14,3	7/24
Nói chồng + đoạn nhiễu	small	30,9	25,7	19,4	2/24



Hình 3.1. *Tỷ lệ lỗi từ (đã chuẩn hoá số) theo điều kiện âm thanh và cấu hình*

Với cấu hình mặc định (small, int8, CPU), WER ở điều kiện sạch là 21,7% và CER là 15,8%. Mức lỗi tăng dần theo nhiễu: khoảng 25–27% ở SNR 20–10 dB và ở nhiễu đông người, lên 36,6% khi nhiễu rất mạnh (SNR 5 dB). Nghĩa là mô hình chịu được nhiễu vừa phải mà không sụp đổ, nhưng chất lượng giảm rõ khi tín hiệu kém.

Phân tích lỗi cho thấy nguồn lỗi lớn nhất là **thuật ngữ tiếng Anh**: ở điều kiện sạch, Whisper small chỉ nhận đúng 2/24 lần xuất hiện thuật ngữ. "PostgreSQL" bị nhận thành "POSTEGARER-ESQUILLER", "Swagger" thành "so gơ", "eSMS" thành "EAS-MS", "MoMo" thành "mô mô". Một phần nguyên nhân nằm ở dữ liệu: giọng tổng hợp đọc thuật ngữ theo cách phát âm tiếng Việt, khác cách nói của người thật. Nguồn lỗi thứ hai là **dấu thanh và các từ gần âm** ("chốt" thành "chất", "đăng nhập" thành "đang nhập", "cuối tuần này" thành "cuối tôm nay"), điển hình của mô hình đa ngôn ngữ cỡ nhỏ với tiếng Việt.

Từ điển thuật ngữ (đưa danh sách thuật ngữ của dự án vào bước giải mã qua tham số hotwords của faster-whisper) cải thiện mạnh độ đúng thuật ngữ: từ 2 lên 11/24 ở điều kiện sạch, từ 7 lên 16/24 ở nhiễu trắng 10 dB và từ 3 lên 12/24 ở nhiễu đông người. Ở hai điều kiện nhiễu, WER giảm khoảng 4–5 điểm phần trăm. Tuy nhiên ở điều kiện sạch, WER tổng thể không giảm (22,4% so với 21,7%) và CER tăng nhẹ, do từ điển đôi khi kéo cả những đoạn không chứa thuật ngữ về phía các từ trong danh sách. Kết luận rút ra là từ điển thuật ngữ đáng dùng khi mục tiêu là giữ đúng tên công nghệ, sản phẩm trong biên bản, và nên được cấu hình theo từng dự án thay vì một danh sách chung.

Mô hình medium giảm WER ở điều kiện sạch từ 21,7% xuống 14,7% (giảm khoảng một phần ba) và CER từ 15,8% xuống 10,9%; ở nhiễu trắng 10 dB, WER giảm từ 26,9% xuống 22,4% và ở nhiễu đông người từ 26,9% xuống 18,3%, nhưng chạy chậm hơn khoảng 2,7 lần trên CPU (xem mục thời gian xử lý). Đây là đánh đổi điển hình giữa độ chính xác và tài nguyên tính toán; với máy chủ có GPU, mô hình medium hoặc lớn hơn là lựa chọn hợp lý hơn.

3.4. Kết quả tách người nói và phát hiện nói chồng

Bảng 3.4. *Kết quả tách người nói (Whisper small)*

Điều kiện	Số người tìm được / thực tế	Độ đúng người nói (%)	Đoạn bị gắn cờ nói chồng
Sạch	4/4	99,5	8 / 35 đoạn (không có nói chồng thật)
Nhiễu trắng 20 dB	4/4	98,9	8 / 37
Nhiễu trắng 10 dB	5/4	94,8	11 / 44
Nhiễu trắng 5 dB	7/4	88,0	10 / 40
Đông người 10 dB	7/4	84,9	8 / 48
Nói chồng + đoạn nhiễu	5/4	93,4	1 / 42 (không trùng vùng nói chồng nào trong 4 vùng)

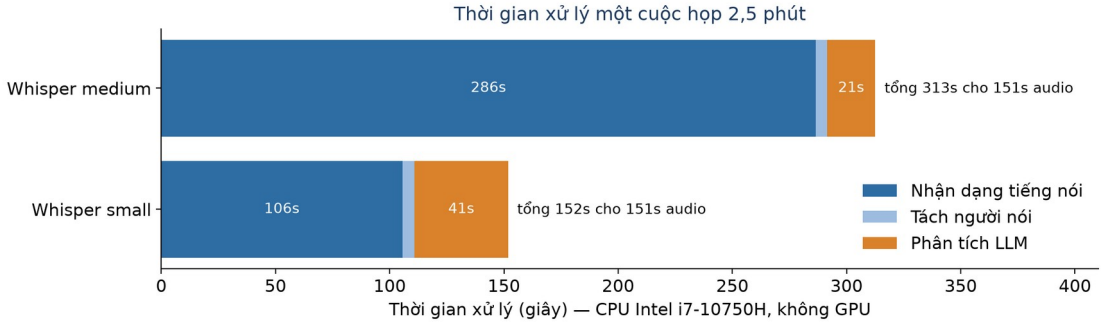
Kết quả ở Bảng 3.4 cho thấy mô-đun tách người nói hoạt động tốt ở điều kiện sạch và nhiễu nhẹ: tìm đúng 4 người và gán đúng người cho 99,5% thời lượng lời nói. Khi nhiễu tăng, mô-đun có xu hướng **tách thừa** (5–7 cụm cho 4 người): một số cửa sổ nhiễu nặng tạo vector đặc trưng lệch khỏi cụm của người nói và hình thành cụm nhỏ riêng. Nhờ bước gộp cụm nhỏ dưới 2 giây và cách ghép tối ưu, độ đúng theo thời gian vẫn giữ ở mức 85–95%, tức là phần lớn lời nói vẫn được gán đúng người; các "người thừa" chiếm ít thời lượng và người dùng có thể gộp lại khi gán tên.

Kết quả **phát hiện nói chồng là không đạt**. Trên bản ghi có 4 vùng nói chồng, mô-đun không gắn cờ đúng vùng nào; ngược lại, trên các bản ghi không hề có nói chồng, 8–11 đoạn vẫn bị gắn cờ. Nguyên nhân nằm ở thiết kế ngưỡng thích nghi (trung vị cộng hai lần độ lệch tuyệt đối trung vị của độ tương đồng với người nói thứ hai): ngưỡng tương đối luôn đẩy phần đuôi của phân bố lên thành cờ, kể cả khi không có nói chồng; trong khi các vùng nói chồng ngắn (1,2 giây) bị cửa sổ 1,5 giây làm loãng. Đây là phát hiện quan trọng của thực nghiệm: chức năng này cần thiết kế lại bằng mô hình phát hiện nói chồng chuyên dụng hoặc ngưỡng tuyệt đối được hiệu chỉnh trên dữ liệu thật, và trước mắt không nên hiển thị cờ "nói chồng" cho người dùng như một tín hiệu tin cậy. Theo kết quả này, cờ đã được tắt mặc định trong hệ thống.

Về **hiện tượng ảo giác**, trong đoạn 8 giây chỉ có nhiễu nền, hệ thống không sinh ra đoạn văn bản nào. Kết quả này phù hợp với các biện pháp đã áp dụng (lọc khoảng lặng bằng VAD, bỏ đoạn bịa giữa khoảng lặng dài, không mượn ngữ cảnh câu trước, lọc câu rác thường gặp).

3.5. Thời gian xử lý, chi phí và hiệu quả tiết kiệm thời gian

3.5.1. Thời gian xử lý



Hình 3.2. *Thời gian xử lý từng bước cho bản ghi 151 giây (CPU, không GPU)*

Hình 3.2 và Bảng 3.5 cho thấy với cấu hình mặc định, hệ thống xử lý bản ghi 151 giây trong khoảng 150 giây: phiên âm chiếm phần lớn (khoảng 106 giây), tách người nói chỉ khoảng 5 giây, phân tích LLM khoảng 41–45 giây. Hệ số thời gian thực của bước phiên âm và tách người nói là 0,73, tức là trên laptop không GPU, hệ thống xử lý nhanh hơn thời gian thực một chút. Với mô hình medium, riêng bước phiên âm mất 287 giây (RTF khoảng 1,9). Ngoại suy tuyến tính, một cuộc họp 60 phút cần khoảng 44 phút xử lý âm thanh với mô hình small hoặc khoảng 2 giờ với mô hình medium trên phần cứng này; trên máy chủ có GPU, thời gian này giảm nhiều lần. Vì toàn bộ xử lý chạy nền và bất đồng bộ, người dùng không phải chờ trước màn hình.

Bảng 3.5. *Thời gian xử lý bản ghi 151 giây ở điều kiện sạch*

Bước	Whisper small	Whisper medium
Nạp mô hình (một lần mỗi tiến trình)	1,8 s	5,5 s
Phiên âm (2 lượt, nhận diện ngôn ngữ)	105,6 s	286,5 s
Tách người nói	5,3 s	5,0 s
Phân tích LLM (DeepSeek-V4.1-Flash)	40,9–44,6 s	21,1–33,9 s
Hệ số thời gian thực (phiên âm + tách người nói)	0,73	1,93

3.5.2. Chi phí

Chi phí LLM cho một lần phân tích bản họp thử nghiệm (khoảng 1.700 token đầu vào và 2.900–5.600 token đầu ra, phần lớn là token suy luận) là khoảng 0,004–0,007 USD. Với cuộc họp 60 phút (khoảng 15.000 token transcript), chi phí ước tính dưới 0,03 USD mỗi cuộc họp. Phiên âm và tách người nói chạy trên máy chủ của tổ chức nên không phát sinh chi phí API.

3.5.3. Ước tính hiệu quả tiết kiệm thời gian

Đề tài chưa có điều kiện đo trên người dùng thật, vì vậy hiệu quả tiết kiệm thời gian dưới đây là **ước tính dựa trên giả định**, cần được kiểm chứng bằng nghiên cứu người dùng ở giai đoạn sau. Giả định cho một cuộc họp 60 phút của nhóm 4–6 người: không có hệ thống, thư ký vừa họp vừa ghi chép, sau họp mất khoảng 30 phút soạn biên bản và khoảng 10 phút tạo, giao công việc trên công cụ quản lý. Có hệ thống, máy tự xử lý nền; trưởng nhóm mất khoảng 10 phút rà soát biên bản nháp và duyệt các đề xuất, mỗi đề xuất đã có sẵn câu trích dẫn để đối chiếu nhanh.

Bảng 3.6. *Ước tính thời gian công sức con người cho một cuộc họp 60 phút*

Công việc	Không có hệ thống	Có MeetingMind
Ghi chép trong lúc họp	Một người ghi liên tục	Không cần (máy ghi âm hoặc đọc phụ đề)
Soạn biên bản sau họp	~30 phút	~5 phút rà soát bản nháp
Tạo và giao công việc	~10 phút	~5 phút duyệt đề xuất
Tổng thời gian sau họp	~40 phút	~10 phút

Theo giả định này, mỗi cuộc họp tiết kiệm khoảng 30 phút công sức sau họp, và người ghi chép được giải phóng để tham gia thảo luận. Với nhóm họp 3 buổi mỗi tuần, mức tiết kiệm khoảng 1,5 giờ mỗi tuần. Thực nghiệm ở mục trước cũng cho thấy bản nháp không hoàn hảo (đặc biệt khi âm thanh kém), nên bước rà soát của con người là bắt buộc chứ không phải hình thức; thiết kế hàng đợi duyệt với câu trích dẫn đi kèm giúp bước này nhanh hơn so với đọc lại toàn bộ transcript.

CHƯƠNG 4. KẾT QUẢ NỘI DUNG VÀ ĐÁNH GIÁ

4.1. Đối chiếu với kết quả cần đạt

Bảng 4.1. *Đối chiếu kết quả Nội dung 3 với đề cương*

Kết quả cần đạt theo đề cương	Kết quả thực hiện	Đánh giá
Triển khai dịch vụ thu âm	Ba đường thu nhận: tải tệp lên MinIO (khử trùng lặp, URL ký), bot ghi âm Google Meet (Playwright, Xvfb, PulseAudio, MediaRecorder), tiện ích Chrome đọc phụ đề	Đạt (bot mới kiểm thử đến bước vào phòng chờ)
API hoàn chỉnh cho xử lý âm thanh	API tải tệp, khởi động phiên âm, mời/dừng bot, nhận phụ đề trực tiếp; điều phối qua RabbitMQ với outbox và chống xử lý lặp	Đạt
Xử lý nhiễu	Lọc khoảng lặng bằng VAD, chống câu bịa của Whisper; không sinh văn bản trong đoạn chỉ có nhiễu; WER tăng từ 21,7% (sạch) lên 26,9% (nhiễu 10 dB)	Đạt một phần (nhiễu rất mạnh 5 dB: WER 36,6%)
Module STT ổn định, chính xác cao trên tiếng Việt	Whisper small trên CPU: WER 21,7%, CER 15,8%; Whisper medium: WER 14,7%; nhận diện ngôn ngữ theo đoạn cho hội thoại xen Anh – Việt	Đạt ở mức dùng được, cần mô hình lớn hơn để đạt độ chính xác cao
(Mở rộng) Tách người nói	Tìm đúng 4/4 người, độ đúng 99,5% ở điều kiện sạch, 85–95% khi có nhiễu	Đạt; đánh dấu nói chồng chưa đạt

4.2. Tồn tại và hướng hoàn thiện

Kết quả nội dung 3 đáp ứng yêu cầu về dịch vụ thu âm và API xử lý âm thanh. Về độ chính xác, cấu hình mặc định (Whisper small chạy CPU) đủ để làm đầu vào cho bước trích xuất có con người rà soát, nhưng chưa đạt mức "chính xác cao" như kỳ vọng của đề cương, chủ yếu do thuật ngữ tiếng Anh và giới hạn phần cứng. Thực nghiệm đã chỉ ra hai hướng cải thiện có hiệu quả đo được: dùng mô hình medium (WER giảm khoảng một phần ba) khi có GPU, và cấu hình từ điển thuật ngữ theo dự án (số thuật ngữ nhận đúng tăng từ 2 lên 11/24). Chức năng đánh dấu nói chồng cần thiết kế lại bằng mô hình phát hiện nói chồng chuyên dụng; kịch bản ghi âm dài của bot sau khi được cho vào phòng cần được kiểm thử trọn vẹn với cuộc họp thật.

TÀI LIỆU THAM KHẢO

[1] Bredin H., Yin R., Coria J. M., Gelly G., Korshunov P., Lavechin M., Fustes D., Titeux H., Bouaziz W., Gill M.-P. (2020), "pyannote.audio: neural building blocks for speaker diarization", *Proceedings of IEEE ICASSP 2020*, pp. 7124–7128.

[2] Chen Y., Zheng S., Wang H., Cheng L., Chen Q., Qi J. (2023), "An enhanced Res2Net with local and global feature fusion for speaker verification", *Proceedings of Interspeech 2023*.

[3] Fireflies.ai (2026), *Fireflies.ai – AI notetaker for meetings*, https://fireflies.ai, truy cập ngày 04/11/2026.

[4] Google (2026), *Chrome Extensions – Manifest V3*, https://developer.chrome.com/docs/extensions, truy cập ngày 04/11/2026.

[5] Hu V. C., Ferraiolo D., Kuhn R., Schnitzer A., Sandlin K., Miller R., Scarfone K. (2014), *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, NIST Special Publication 800-162.

[6] Ji Z., Lee N., Frieske R., Yu T., Su D., Xu Y., Ishii E., Bang Y. J., Madotto A., Fung P. (2023), "Survey of hallucination in natural language generation", *ACM Computing Surveys*, 55(12), pp. 1–38.

[7] k2-fsa (2026), *sherpa-onnx: Speech-to-text, text-to-speech, speaker diarization using next-gen Kaldi with onnxruntime*, https://github.com/k2-fsa/sherpa-onnx, truy cập ngày 04/11/2026.

[8] Kleppmann M. (2017), *Designing Data-Intensive Applications*, O'Reilly Media.

[9] Levenshtein V. I. (1966), "Binary codes capable of correcting deletions, insertions, and reversals", *Soviet Physics Doklady*, 10(8), pp. 707–710.

[10] Microsoft (2026), *Playwright documentation*, https://playwright.dev, truy cập ngày 04/11/2026.

[11] Myśliwiec K. (2026), *NestJS documentation*, https://docs.nestjs.com, truy cập ngày 04/11/2026.

[12] Otter.ai (2026), *Otter.ai – AI meeting assistant*, https://otter.ai, truy cập ngày 04/11/2026.

[13] Park T. J., Kanda N., Dimitriadis D., Han K. J., Watanabe S., Narayanan S. (2022), "A review of speaker diarization: Recent advances with deep learning", *Computer Speech & Language*, 72, 101317.

[14] RabbitMQ (2026), *RabbitMQ documentation: Reliability guide, consumer acknowledgements and publisher confirms*, https://www.rabbitmq.com/docs, truy cập ngày 04/11/2026.

[15] Radford A., Kim J. W., Xu T., Brockman G., McLeavey C., Sutskever I. (2023), "Robust speech recognition via large-scale weak supervision", *Proceedings of the*

40th International Conference on Machine Learning (ICML 2023), PMLR 202, pp. 28492–28518.

[16] Richardson C. (2018), *Microservices Patterns: With Examples in Java*, Manning Publications.

[17] Sandhu R. S., Coyne E. J., Feinstein H. L., Youman C. E. (1996), "Role-based access control models", *IEEE Computer*, 29(2), pp. 38–47.

[18] Snyder D., Garcia-Romero D., Sell G., Povey D., Khudanpur S. (2018), "X-vectors: Robust DNN embeddings for speaker recognition", *Proceedings of IEEE ICASSP 2018*, pp. 5329–5333.

[19] SYSTRAN (2026), *faster-whisper: Faster Whisper transcription with CTranslate2*, https://github.com/SYSTRAN/faster-whisper, truy cập ngày 04/11/2026.

[20] Tactiq (2026), *Tactiq – AI meeting transcripts for Google Meet, Zoom and Teams*, https://tactiq.io, truy cập ngày 04/11/2026.

[21] W3C (2021), *WebRTC 1.0: Real-Time Communication Between Browsers*, W3C Recommendation, https://www.w3.org/TR/webrtc/.